

DeepSeek对OpenCode 自动化代码审查流程实现的建议

核心思路：为不同任务指定不同模型

要实现这个流程，关键在于根据任务来切换模型，大致可以分为“编排”和“Agent 化”两类方案：

- **外部编排**：用一个外部“大脑”（如脚本或工作流工具）来调度 OpenCode，让它按顺序用不同的模型执行任务。
- **Agent 化**：在 OpenCode 内部创建一个“审查 Agent”，并让它专门使用 GPT-5.4 模型，然后集成到主流程中。

方案一：使用 `aworkflow` 进行代码编排（最贴近你需求）

`aworkflow` 是一个 Python 工作流引擎，它的核心功能就是让你能够编排不同 AI Agent 的执行流程。它不直接调用 API，而是像“指挥官”一样去调用你本地已有的 Agent 命令行工具，比如 OpenCode 和 Copilot。

你可以这样用：

1. **安装**： `uv tool install aworkflow`
2. **安装 Skill**：为了让 `aworkflow` 能和 OpenCode 更好地协作，你需要安装它提供的 skill 文件： `aflow install-skills`。它会自动将 skill 文件复制到 `~/.config/opencode/skills` 目录下。
3. **准备计划文件**：创建一个 Markdown 文件（比如 `code_review_plan.md`），把你的流程描述清楚：

markdown

Plan: 代码生成与审查自动化

[] 阶段 1: Kimi 2.5 生成初始代码

- ☐ 使用 OpenCode 和 Kimi 2.5 模型实现[你的功能描述]。
- ☐ 将生成的代码保存到[你的代码文件路径]。

[] 阶段 2: GPT-5.4 审查代码

- ☐ 使用 GitHub Copilot (内含 GPT-5.4) 审查上一步生成的代码。
- ☐ 将审查报告（改进建议）保存为 `review_suggestions.md`。

[] 阶段 3: Kimi 2.5 修复代码

- ☐ 使用 OpenCode 和 Kimi 2.5 模型读取 `review_suggestions.md` 文件。
- ☐ 根据建议修改和修复[你的代码文件路径]。

4. 运行: `aflow run code_review_plan.md`

`aworkflow` 会按顺序执行这些步骤，并管理好中间产生的文件。这个方案的好处是思路清晰，容易上手，并且完全自动化了文件交接环节。

方案二：打造专属的“审查 Agent” (灵活且可复用)

这个方法更“OpenCode 原生”，就是在 OpenCode 内部创建一个专门的 Agent，让它来做代码审查，并通过**自定义指令（Rules）** 来实现流程自动化。

实施步骤：

1. **创建审查 Agent**：在项目的 `.opencode/agents/` 目录下新建 `gpt-reviewer.md` 文件，内容如下：

markdown

description: 一个专门负责审查代码并提供改进建议的 Agent

provider: github-copilot

model: gpt-5.4

身份

你是一名资深的代码审查专家，你的任务是审查提供的代码文件，并输出一份详细的改进建议报告。

workflow

1. 仔细阅读和分析用户提供的代码文件。
2. 识别代码中可能存在的逻辑错误、性能瓶颈、安全隐患或不符合最佳实践的地方。
3. 将所有发现的问题和改进建议整理成一份结构清晰的 Markdown 报告。
4. 将报告保存为 `review_suggestions.md` 文件。

输出格式

你的输出应该只包含审查报告的内容，不要有任何额外的对话。

2. **集成到自动化流程**: 在项目根目录创建一个 `AGENTS.md` 文件, 在其中编写一个 `/workflow` 指令来串联整个过程:

markdown

项目自动化指令

`/workflow:代码迭代`

请严格遵循以下步骤执行任务:

1. **生成**: 首先, 作为主 Agent, 你需要使用 `kimi-k2.5` 模型, 根据用户需求生成代码并写入文件。
2. **审查**: 然后, 调用 `@gpt-reviewer` Agent, 将刚才生成的代码文件作为输入, 生成 `review_suggestions.md` 报告。
3. **修复**: 最后, 你 (主 Agent) 再次使用 `kimi-k2.5` 模型, 读取 `review_suggestions.md` 文件, 并根据其中的建议修复代码。

使用示例

用户输入 `/workflow:代码迭代` 实现一个用户登录接口, 你应该立即开始执行上述三个步骤。

这种方案的可定制性很强, 除了自定义 Agent, 你还可以利用 `oh-my-opencode-slim` 等现成的插件来快速搭建 Agent 团队。

⚙️ 其他备选方案

- **编写 Shell 脚本**: 如果不想引入新工具, 可以直接用 Shell 脚本串联 `opencode run` 命令。OpenCode 支持非交互式的 `run` 模式。bash

1. Kimi 生成代码

`opencode --model opencode-go/kimi-k2.5 run "请为我生成一个[功能]的代码..."`

2. GPT-5.4 审查 (注意: 这里需要根据你 Copilot CLI 的实际命令调整)

`copilot suggest "审查刚刚生成的代码, 输出到 review_suggestions.md"`

3. Kimi 修复代码

`opencode --model opencode-go/kimi-k2.5 run "请读取 review_suggestions.md, 并修复代码..."`

- **使用编排工具**: 如果你觉得在 OpenCode 内部写规则不够直观, 也可以选择专门的 AI Agent 编排工具来管理流程, 比如 `@pruddiman/dispatch` 或 `opencode-autopm`。

💡 总结与建议

方案	核心工具	优点	缺点	适用场景
外部编排	aworkflow	思路清晰，易于调试，不侵入 OpenCode 配置	依赖外部工具，灵活性相对受限	流程固定、明确，希望快速实现自动化的场景
Agent 化	opencode + AGENTS.md	高度集成、可复用、可自定义性强	配置稍复杂，需要学习 Agent 编写	流程复杂多变、希望构建可复用自动化能力的场景
其他方案	Shell 脚本 / 编排工具	脚本方案简单直接；编排工具功能全面	脚本方案扩展性差；编排工具学习成本高	脚本方案适用于一次性任务；编排工具适用于大型项目

- **建议从 aworkflow 开始：**对于你描述的“代码-审查-修复”这一典型流程，aworkflow 方案最简单直接，容易看到效果。
- **进阶到 Agent 化：**当你熟悉 OpenCode 并希望有更高自定义能力时，可以尝试创建专属的审查 Agent 和自动化指令。
- **按需使用脚本：**对于临时、简单的任务，用 Shell 脚本快速运行一下也很方便。